

AMSS 2026 — Lecture 3: Testable Specs & TDD-as-Spec

Traian-Florin Șerbănuță

2026

Today's Agenda

1. Frame: from requirements (Week 2) to testable specs
2. Live demo: is your spec testable?
3. Tests as the executable form of requirements
4. How AI-written tests fail
5. The vague-spec loop
6. Traceability: use case $\rightarrow$ scenario $\rightarrow$ test

Recap: Architect and Critic

- ▶ **Architect / director:** drive AI through the software development lifecycle (SDLC).
- ▶ **Critic / reviewer:** read AI's output and name what's wrong or missing.

Last week (Week 2) you drove AI to produce a requirements doc — and critiqued it. Today: can you *test* what you wrote?

A Requirement That Looks Fine — Until You Test It

From last week's bike-sharing doc:

"Users are charged for renting a bike."

What would a test assert? `fare(60) == ?`

You can't fill in the blank — the spec never says the price. **That gap is invisible in prose and obvious in a test.**

Four Threads in Today's Lecture

1. A test is the executable form of a requirement.
2. AI writes tests too — and they fail in their own ways.
3. The loop: if AI can't write a passing test from your spec, your spec is too vague.
4. Traceability: use case $\rightarrow$ scenario $\rightarrow$ test.

The Literacy Floor: Critique, Rationale, Traceability

From Week 1: in the oral defense, *unaided*, you must demonstrate:

- ▶ **Critique** — read & critique AI-generated artifacts on the spot.
- ▶ **Rationale** — articulate why you directed AI a certain way.
- ▶ **Traceability** — defend the trace across your project.

Today drills Critique on *tests*, and extends Traceability: the test is the new end of the trace.

Demo: Bike-Sharing Fare — Is Your Spec Testable?

Live: drive AI to write a test for a requirement from last week. Watch what price it invents.

Prompt to AI: *“Here’s a requirement from last week’s bike-sharing app: ‘Users are charged for renting a bike.’ Write a pytest test for the fare calculation.”*

A Test Is a Requirement You Can Run

- ▶ A **requirement** says what the system should do.
- ▶ A **test** says it in a form the machine checks for you.
Same intent. One is prose; the other fails loudly when violated.

Acceptance Criterion = Assertion

Prose acceptance criterion:

"A 60-minute rental costs €3.00."

The same thing, executable:

```
def test_paid_ride_charges_per_minute():  
    assert fare(60) == 3.00
```

The per-minute rate is what makes €3.00 the expected total: the first 30 min are free, then €0.10/min for 30 min = €3.00 (under the €5 cap). The assertion *is* the acceptance criterion — you just can't hand-wave it.

If You Can't Write the Assertion, the Spec Is Underspecified

Try to test “users are charged for renting a bike”:

```
def test_fare():  
    assert fare(60) == ???    # the spec never said
```

No number → no test → the requirement isn't done yet.

Two Kinds of Spec: Prose and Executable

	Prose spec	Executable spec (test)
Readable by	humans	humans + machine
Fails when violated?	silently	loudly
Pins exact behaviour?	optional	required

Both are specs. The test is the one you can't fake.

AI Writes Tests Too — and They Fail in Their Own Ways

You critiqued AI's *requirements* last week. Now critique AI's *tests*.

A test can be green and still be worthless. The critic question is always:

“Does this test fail for a wrong implementation? Does it pin the behaviour I actually want?”

Test Failure #1: Invented Assumption

Definition: the test hard-codes a value the spec never stated.

```
def test_fare():  
    assert fare(60) == 6.0    # where did €6 come from?
```

Critique: *“My spec never said €0.10/min. The AI guessed — and a teammate’s AI would guess differently.”*

Test Failure #2: Trivially-Passing Test

Definition: the assertion holds for almost any implementation.

```
def test_fare():  
    assert fare(10) >= 0    # true for 0, 999, anything
```

Critique: *“Does this fail for a wrong implementation? If not, it proves nothing.”*

Test Failure #3: Tautological Test

Definition: the expected value is computed the same way the code computes it.

```
def test_fare():  
    assert fare(60) == (60 - 30) * 0.10    # mirrors the im
```

Critique: *"If the code and the test share the same bug, the test still passes."*

Two More: Missing Boundary & Faithful-to-a-Wrong-Spec

- ▶ **Missing boundary:** no test at exactly 30 min, at the cap, or at 0. The interesting bugs live at the edges.
- ▶ **Faithful to a wrong spec:** the test correctly encodes a spec that is itself wrong (e.g. no cap $\rightarrow$ unbounded charge). The test is right; the spec isn't.

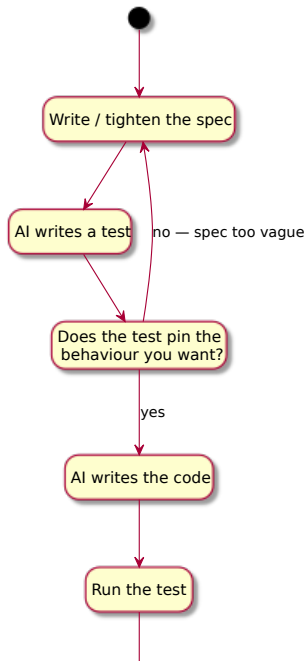
Your Turn: Name the Failure Mode

Al wrote this test for the fare spec:

```
def test_fare():  
    assert fare(45) == 1.50
```

With your neighbour (60s): which of the five failure modes is this?
Be ready to name your critique question — then we reveal.

The Loop



The Mantra

If AI can't write a passing test from your spec, your spec is too vague.

You watched it live: the vague fare spec produced a guessed test; the tightened spec produced a test that pins the behaviour.

Worked Example: The Fare Spec in Two Passes

Pass 1 (vague): “Users are charged for renting a bike.” → AI invents €0.10/min. The test encodes a guess.

Pass 2 (tightened): “Free first 30 min, then €0.10/min, capped at €5.” → AI writes tests for 20 min, 30 min, 60 min, and the cap. Each pins a real decision.

When Does the Loop Stop?

Not when the code is “done.”

The loop stops when the test stops surprising you — when every assertion is one you'd have written by hand and stand behind.

The Human Closes the Loop

AI will happily generate plausible tests forever.

What it can't do: decide whether a test expresses *what you actually want*. That judgment — the architect-and-critic call — is yours, and it's what the oral defense checks.

The Chain Gains a Tail

Last week's chain:

requirement → *use case*

This week it grows an executable end:

requirement → *use case* → ***scenario*** → ***test***

Scenario = A Concrete Instance of a Use Case

- ▶ **Use case:** “Rent a bike” (one user goal).
- ▶ **Scenario:** “A 60-minute rental.” (one concrete path through it).
- ▶ **Test:** `test_paid_ride_charges_per_minute` (that scenario, runnable).

A use case has many scenarios; each interesting scenario becomes a test case.

Test = Scenario Made Runnable

The test is the binding that makes traceability *verifiable*, not just documented.

A documented trace says “this requirement is covered.” A test proves it — and breaks loudly when the link rots.

Broken Links (a preview of Week 10)

A test that asserts behaviour **no requirement asked for** is an orphan — just like a fabricated requirement with no user need.

Week 10 makes finding broken links across the whole trace its own skill.

Your Project: The test-driven-development (TDD)-with-AI Reflection

The loop you saw today *is* a required project deliverable:

- ▶ Run it on **one feature** of your slice: spec → AI tests → AI code → run.
- ▶ Document what the failures revealed about your spec.
- ▶ Code is a byproduct. The **reflection** is what gets read.

Next Week: Class Diagrams (Week 4)

Once a requirement is pinned by a test, the next question is: *what objects realise it?*

- ▶ **Week 4:** class diagrams — the structure behind the behaviour.
- ▶ **Lab 2** (Week 4's slot): drive AI to produce a class diagram from a spec, iterate, log what changed.

Critique, Rationale, Traceability — The Through-Line

Today you drilled:

- ▶ **Critique** — read AI-written tests and named how they fail.
- ▶ **Traceability** — the test is the *end* of the trace; if you can't trace a test back to a requirement, the chain is broken.

Rationale (articulating *why* you directed AI a certain way) lands in your project narrative.

That's It For Today

- ▶ Next lecture (Week 4): class diagrams.
- ▶ Your project: run the spec $\rightarrow$ test $\rightarrow$ code loop on one feature; write the reflection.

Questions?